

# round3\_guard\_residue\_fix

## BOB-168 round 3 — the reviewer's MINOR closed, with its paired mutation

Revision: 1 Last modified: 2026-08-23T11:45:00Z

### The finding (reviewer, round 2, NO-GO)

Four pre-run mutation lines in tests/unit/test\_run\_all\_challenges\_missing\_entry.sh consumed a possibly-empty \$R **before** run\_runner's guard could refuse: chmod (case 3), printf > (cases 4 and 4b), rm -f (case 4b). On a failed fixture they operate on /submodules/...-rooted paths.

The reviewer's prose named four lines; its fix summary said "three-line change". I trusted the lines, not the count (\$11.4.201 — a count is a lead, the lines are the finding) and measured: **four** dereferences, at 149, 167, 182, 183.

### The fix — one predicate, not four inline copies

Rather than prefixing each line with a raw `[[ -n "$R" ]]`, the "is this root usable" test is extracted into ONE definition consumed by run\_runner AND by every pre-run mutation line:

```
usable_root() { [[ -n "${1:-}" ]&& -f "${1}/scripts/  
run_all_challenges.sh" ]]; }
```

run\_runner now calls it instead of restating it. A second, weaker inline notion of “valid root” is exactly how this residue arose (§11.4.251), and a future case that adds a fifth mutation line inherits the same predicate rather than inventing one. That is the API-shape rung rather than four copies of a lint-level guard (§11.4.241).

## Evidence

### GREEN — 10/10, unchanged

RESULT: 10 passed, 0 failed                      exit=0

### The fail-closed path, with the precondition actually reached

**First attempt was INVALID and is recorded as such (§11.4.199).** I ran the drift probe from the scratchpad; the test derives PROJECT\_ROOT from BASH\_SOURCE (:36-38), so it resolved to the scratchpad, RUNNER did not exist, and the run bailed at preflight — FAIL: runner missing, 0 passed, 1 failed. Zero anti-replica trips: **the mutation lines were never reached, so that run proved nothing about the guard.** Caught by reading the tail rather than the counters.

Redone with the probe inside tests/unit/ so PROJECT\_ROOT resolves correctly:

|                                                     | pre-guard (my fix<br>reverted) | post-<br>guard |
|-----------------------------------------------------|--------------------------------|----------------|
| anti-replica trips —<br><b>precondition reached</b> | 5                              | 5              |
| root-path write/rm attempts                         | 3                              | 0              |
| RUN: lines (real bank executed)                     | 0                              | 0              |
| exit                                                | 1                              | 1              |

Pre-guard residue, verbatim:

```
chmod: cannot access '/submodules/challenges/challenges/scripts/  
scaling_horizontal_challenge.sh'  
line 167: /submodules/challenges/challenges/scripts/  
bluff_scanner_challenge.sh: No such file or directory  
line 182: /submodules/challenges/challenges/scripts/  
bluff_scanner_challenge.sh: No such file or directory
```

Same precondition on both sides, three attempts before and zero after: the guard is load-bearing, not decoration.

**Honest asymmetry worth stating:** only **three** of the four lines leave a trace. `rm -f` on a nonexistent path succeeds silently, so line 183 — the `rm -f` at a `/`-rooted path derived from an empty variable, the canonical §11.4.252 shape and the most dangerous of the four — produces *no* evidence either way. Its guard is verified by construction (same predicate, same call site pattern) and by the absence of the two `printf` attempts from the same case body, not by its own observable. A reader should not mistake “3 of 4” for “one line unfixed”.

## Safety of the demonstration

Run as uid 1000 (non-root, so root-path writes are refused by the OS regardless), `/submodules` verified absent before and after both runs, and each probe file was created, run, and deleted inside a single command window with a `trap ... EXIT` — the ab41bab auto-commit hazard (BOB-068) was live during this work.

## Owed to round 3

The reviewer’s other four axes were already GREEN in round 2 (B-1 closed, M5 dead at case 4b, its seventh reviewer-authored mutation set all caught, export twins fresh and correctly scoped, staging empty). This closes the one remaining MINOR.

## The declared weak spot is CLOSED — reviewer round 3, syscall-level observable

The section above states that line 183’s `rm -f` has no observable in either direction and rests on a by-construction argument. **That boundary no longer holds — the reviewer closed it**, and the closure belongs in this durable record rather than only in the review thread.

Method: `strace -f -e trace=unlink,unlinkat`, with the instrument proven alive in the same run before its zero was trusted (§11.4.201(7)(b)).

Control needle first — establishing that the shell layer silences what the syscall layer sees: `rm -f` on a nonexistent `/`-rooted path exits 0 with no output, yet emits

```
unlinkat(..., "/submodules/.../scaling_horizontal_challenge.sh",  
0) = -1 ENOENT
```

Both directions, same drift precondition:

|                                                                              | pre-guard | post-guard |
|------------------------------------------------------------------------------|-----------|------------|
| unlinkat on a /<br>submodules path (line<br>183's own evidence)              | 1         | 0          |
| unlink-class syscalls<br>from fixture cleanup<br>(instrument-alive<br>proof) | —         | 16         |

The post-guard zero is a *seen* zero, not a blind one: 16 unlink-class syscalls were captured in the same trace, so the instrument was demonstrably watching the right call on the right path when it reported none against `/submodules`.

**“3 of 4 observable” is now 4 of 4.** No by-construction argument remains load-bearing anywhere in this change.

The reviewer also corrected its own count: its round-2 summary said “three-line change” because it mentally collapsed case 4b’s two lines into one edit; four is right and no guard here is redundant. And it accepted `usable_root()` as a strengthening rather than scope creep, on three verified grounds — it is strictly stronger than the prescribed `[ [ -n "$R" ] ]` (it also requires the runner file, i.e. that `make_root` completed), it has no over-blocking channel (any root `make_root` echoes contains the runner by construction, confirmed by GREEN 10/10), and one predicate shared by all five consumers kills the divergent-notion channel that produced the residue.

**Round 3 verdict: GO, zero findings, zero warnings (§11.4.134 satisfied).**